

PROJECT REPORT

Student Management System

Applicant Name	Nishi Chauhan
Application ID	KTS020260715709
Internship Domain	Python Development Internship
Organization	Kinetrexa Software Private Limited
Duration	24 July 2026 - 15 August 2026
Task	Task 1: Student Management System

1. Introduction

This report documents the design, development, and testing of the Student Management System, developed as Task 1 of the Python Development Internship at Kinetrexa Software Private Limited. The objective of this project was to build a console-based application capable of managing student records efficiently, while demonstrating core Python concepts including Object-Oriented Programming (OOP), file handling, and exception handling.

2. Objective

To design and implement a Python application that allows users to add, view, search, update, delete, and filter student records through a menu-driven interface, with all data persisted to disk so that it is not lost when the program is closed.

3. Features Implemented

- Add Student - create a new student record with roll number, name, age, course, and marks
- View All Students - display every stored student record
- Search Student - locate a specific student by roll number
- Update Student - modify an existing student's details, retaining old values if left blank
- Delete Student - remove a student record permanently
- Filter Students - filter records by course, or by a minimum marks threshold
- Persistent Storage - all data is saved to a JSON file and automatically reloaded on the next run
- Exception Handling - invalid input and missing or corrupted data files are handled gracefully without crashing the program

4. Technologies Used

Language	Python 3
Data Storage	JSON (built-in json module)
Core Concepts	Object-Oriented Programming, File Handling, Exception Handling
Version Control	Git and GitHub

5. Project Structure

The project follows a modular structure, with responsibilities divided across three files instead of a single monolithic script:

```
student_management_system/  
|-- student.py    # Student class (OOP blueprint)  
|-- storage.py    # Save/load functions for JSON persistence  
|-- main.py       # CRUD logic and menu-driven entry point  
|-- students.json # Auto-generated data file  
`-- README.md
```

5.1 student.py

Defines the **Student** class using OOP. Each student is represented as an object with attributes roll_no, name, age, course, and marks. The class includes a display() method to print a student's details, and to_dict() / from_dict() methods that convert a Student object to and from a plain dictionary, enabling it to be stored in and retrieved from a JSON file.

5.2 storage.py

Handles all file I/O. The load_students() function reads students.json at startup and reconstructs Student objects from it, safely returning an empty list if the file does not exist or is corrupted. The save_students() function writes the current list of students back to the file every time the data changes.

5.3 main.py

Contains the five CRUD functions (add_student, view_students, search_student, update_student, delete_student), the filter_students function, and the main() function that runs a menu-driven loop until the user chooses to exit. Every function that accepts numeric input is wrapped in a try/except block to handle invalid entries gracefully.

6. Sample Output

```
===== STUDENT MANAGEMENT SYSTEM =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Filter Students
7. Exit
=====

Enter your choice (1-7): 2
Roll No: 101 | Name: Nishi Chauhan | Age: 20 | Course: Python Development | Marks: 92.0
```

7. Testing

The application was manually tested end-to-end for every feature. All operations were verified to work correctly, including edge cases such as invalid numeric input and searching for a non-existent roll number.

Test Case	Result
Add a new student record	Passed
View all student records	Passed
Search student by roll number	Passed
Update existing student details	Passed
Delete a student record	Passed
Filter students by course	Passed
Filter students by minimum marks	Passed
Data persists after closing and reopening	Passed
Invalid input handling (non-numeric entry)	Passed

8. Challenges Faced

- Understanding Python's indentation-based structure while writing the class, which initially caused a few `NameError` and `AttributeError` issues, resolved by carefully aligning code blocks.
- Ensuring that importing the `Student` class into other modules did not unintentionally re-run test code, solved using the `if __name__ == "__main__":` guard.
- Designing a persistence layer that fails gracefully if the data file is missing or corrupted, rather than crashing the program.

9. Conclusion

The Student Management System successfully meets all the requirements specified in the internship task assignment. It demonstrates a solid understanding of Object-Oriented Programming, file handling, exception handling, and modular code design in Python. The project has been version-controlled with Git and is publicly available on GitHub, along with complete source code and documentation.

10. Deliverable Links

GitHub Repository	https://github.com/NishiChauhan26/student-management-system
--------------------------	---

Submitted by: Nishi Chauhan

Python Development Intern, Kinetrexa Software Private Limited